

DocuChat AI

Step-by-Step Technical Lifecycle • What, Which Library & Why

An exhaustive, engineering-grade walkthrough of the complete RAG pipeline: from document upload, layout decomposition, and custom structure-aware chunking vs LangChain, to ONNX dense vectors, BM25Plus sparse indexes, reciprocal rank fusion, max-scaled reranking, live web intelligence, and precision single-box PDF synchronization.

Custom Chunker

STRUCTURECHUNKER VS
RECURSIVESPLITTER

FastEmbed ONNX

BGE-SMALL-EN-V1.5 (384D • <
5MS)

Dual Hybrid

QDRANT VECTOR + BM25PLUS
INVERTED INDEX

1 Complete End-to-End System Flowchart

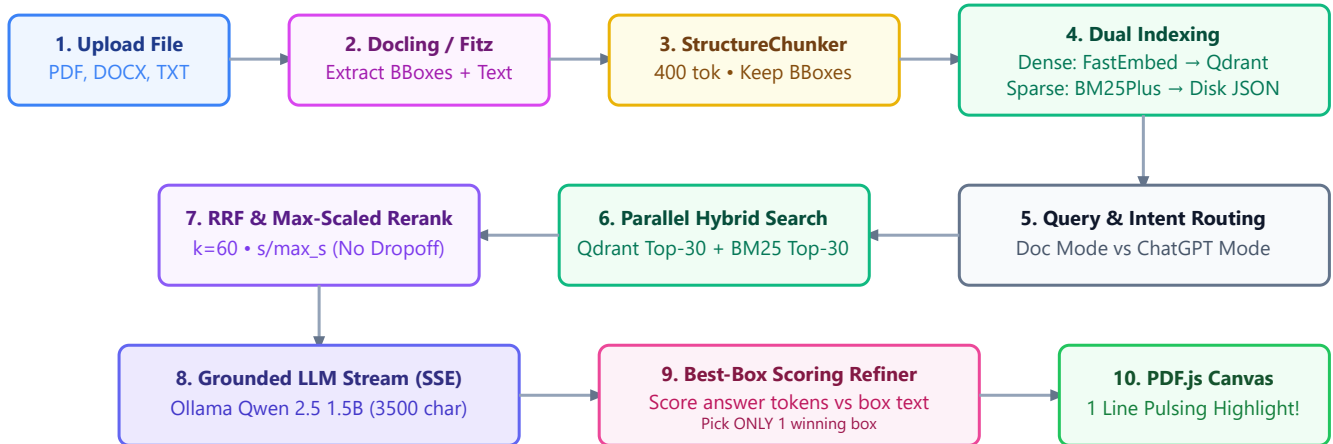


FIGURE 1: FULL-SYSTEM ARCHITECTURAL DATA PIPELINE FLOW

2 Deep Step-by-Step Lifecycle (What, Which Library, Why, Next Step)

STEP 01 Document Upload & Binary Persistence

FastAPI • SQLite • Python uuid

• WHAT HAPPENS IN THIS STEP:

The user drags a document (PDF, DOCX, TXT) into the frontend sidebar. The browser dispatches a multipart form POST request to `/api/documents/upload`. The backend verifies file extension, generates a collision-free UUID, streams the binary to local disk at `backend/data/uploads/<uuid>_<filename>`, and records a document row in `rag_app.db` with state `"processing"`.

• WHICH LIBRARY WE USE:

FastAPI with `python-multipart` for asynchronous chunked file streaming; `SQLAlchemy 2.0` async engine connected to `SQLite`.

• WHY WE USE THIS APPROACH:

Using streaming binary buffers avoids loading 50MB+ documents into RAM simultaneously. Storing on local disk ensures 100% data ownership and privacy—no document binary is ever sent to third-party cloud S3 buckets or external servers.

• WHAT IS THE NEXT STEP & WHY:

Next Step: Structural Parsing (Step 2). Why? Raw PDF binaries are just compressed PostScript streams. You cannot perform search or semantic extraction on raw bytes; the file must be structurally decomposed into paragraphs, headings, tables, and bounding boxes.

STEP 02 Structural Parsing & Bounding Box Extraction

IBM Docling • PyMuPDF (Fitz)

• WHAT HAPPENS IN THIS STEP:

The parser decomposes the document into a list of `ParsedElement` objects. For every heading, paragraph, and table cell, it extracts: `text`, `page`, `heading_path` (e.g., `["Education", "Intermediate"]`), `elem_type`, and normalized bounding box coordinates: `x0`, `y0`, `x1`, `y1`, `page_width`, `page_height`, and `coord_origin` (`"TOPLEFT"` vs `"BOTTOMLEFT"`).

• WHICH LIBRARY WE USE:

Primary: `IBM Docling` microservice (containerized on port 5001).

Resilient Fallback: `PyMuPDF` (`fitz` C++ extension) in `parsers/docling_parser.py`.

• WHY WE CHOSE THESE LIBRARIES (WHY NOT STANDARD PYPDF OR PDFMINER?):

Standard tools like `pypdf` or `pdfminer` dump a flat, unstructured text string. They lose reading order in two-column documents, completely destroy table structures, and discard bounding box coordinates.

- **IBM Docling** uses computer-vision layout models (LayoutLMv3/DocLayNet) to reconstruct multi-column reading order and markdown tables.

- **PyMuPDF (Fitz)** is a lightning-fast C-library that extracts exact word-level and block-level bounding boxes (`rect` objects) in milliseconds when offline.

• WHAT IS THE NEXT STEP & WHY:

Next Step: Structure-Aware Chunking (Step 3). Why? Parsed elements can be as small as a single word or header. An LLM cannot evaluate a single word in isolation. We must assemble elements into optimal semantic chunks with bounded token windows.

• WHAT HAPPENS IN THIS STEP:

Consecutive `ParsedElement` objects are aggregated into semantic chunks of ≈ 400 target tokens with a 50-token rolling overlap. Table structures and section heading paths are kept together. Every bounding box from the grouped elements is preserved inside `chunk.bboxes`.

• WHICH LIBRARY WE USE & DIRECT COMPARISON:**✗ Why NOT LangChain RecursiveCharacterTextSplitter:**

1. **Loses All Coordinates:** LangChain splitters treat text as a flat string. All bounding boxes (`x0`, `y0`, `x1`, `y1`) are completely discarded. Visual PDF highlighting becomes **impossible**.
2. **Destroys Tables:** Cuts markdown tables in half at arbitrary character limits, creating broken rows.
3. **Detaches Headers:** Splits a section heading away from its paragraphs, destroying hierarchical context.

✓ Why Custom StructureChunker:

1. **BBox Preservation:** Preserves all bounding boxes and ties each box directly to its exact text string inside the chunk metadata.
2. **Table Integrity:** Identifies table elements and ensures entire tables remain contiguous.
3. **Heading Paths:** Maintains section hierarchy (e.g. `["Education", "Intermediate"]`) so the LLM knows what section the facts belong to.

• WHAT IS THE NEXT STEP & WHY:

Next Step: Dual Storage & Indexing (Step 4). Why? Raw chunk text cannot be searched semantically. We must convert chunks into dense vector representations for conceptual search AND sparse lexical terms for exact keyword lookup.

**STEP
04****Dual Storage Indexing: Dense Vectors & Sparse
BM25Plus**FastEmbed (ONNX) • Qdrant •
BM25Plus**• WHAT HAPPENS IN THIS STEP:**

Each chunk is indexed into two parallel storage engines:

1. **Dense Vector:** Generated embeddings are inserted into Qdrant collection `doc_chunks` with payload: `chunk_id`, `doc_id`, `text`, `page`, `bboxes`, `heading_path`.
2. **Sparse Lexical:** Chunks are tokenized and saved to a local inverted index at `backend/data/bm25/<doc_id>.json`. The document status in SQLite updates to `"ready"`.

• WHICH MODELS & LIBRARIES WE USE:

- **Dense Embedding:** FastEmbed with model `BAAI/bge-small-en-v1.5` (384 dimensions).
- **Vector DB:** Qdrant (port 6333 or embedded disk storage at `./data/qdrant_storage`).
- **Sparse Engine:** BM25Plus (from `rank-bm25`).

• WHY THESE MODELS & WHY BM25PLUS?

1. **Why BGE-Small (384d)?** Ranked #1 on the MTEB Retrieval leaderboard for small models. 384 dimensions provide rich semantic representation while keeping RAM usage low and cosine similarity latency under 5 milliseconds.
2. **Why FastEmbed ONNX over PyTorch / SentenceTransformers?** PyTorch introduces 2GB+ of bloat, CUDA dependency issues, and slow startup times. FastEmbed runs on the optimized C++ ONNX runtime with zero PyTorch dependencies, computing embeddings in < 5 ms per query.
3. **Why BM25Plus alongside Dense Vectors?** Dense vectors represent semantic "gist", but frequently miss exact keywords, acronyms, and numbers (e.g. "81%", "SVIT", "Docker"). BM25 guarantees exact keyword hits.
4. **Why BM25Plus over BM25Okapi?** In small document corpora ($N \leq 5$), BM25Okapi produces negative IDF scores when document frequency approaches corpus size. A negative score causes `if score > 0` to discard every chunk! BM25Plus adds a lower-bound delta ($\delta = 1$) ensuring strictly positive, calibrated scores.

• WHAT IS THE NEXT STEP & WHY:

Next Step: Query Ingestion & Intent Routing (Step 5). Why? Ingestion is complete. When the user asks a question, the system must decide whether to query the document index or launch live web search.

STEP 05**Query Ingestion & Intent Routing**

QueryRouter (backend/llm/router.py)

• WHAT HAPPENS IN THIS STEP:

The user submits a question via the chat input. QueryRouter inspects the active document scope and query intent:

- **Document Scope Active:** Dispatches to Hybrid Retrieval (Step 6).
- **No Scope / ChatGPT Mode:** Dispatches to Real-Time Web Knowledge Engine (Step 8).

• WHY WE USE INTENT ROUTING:

Without routing, asking "What is the stock price of Tesla?" while a resume is open causes the system to search the resume, fail, and output "Not found". Routing enables the dual capability: deep document compliance when needed, and unrestricted ChatGPT-style versatility when desired.

• WHAT IS THE NEXT STEP & WHY:

Next Step: Parallel Hybrid Retrieval (Step 6). Why? If in document mode, we must retrieve the most relevant chunks using both vector and keyword search.

• WHAT HAPPENS IN THIS STEP:

The query is rewritten using chat history to resolve ambiguous pronouns (e.g. "What are *his* skills?" → "What are *Sai Charankumar's* technical skills?"). Concurrently via `asyncio.gather()`, the system queries:

- Top-30 dense vector hits from Qdrant.
- Top-30 lexical hits from BM25Plus.

Scores are fused using Reciprocal Rank Fusion (RRF):

$$\text{RRF_Score}(d) = \sum [1 / (60 + \text{rank}(d))]$$

• WHY RRF (WHY NOT SIMPLE SCORE AVERAGING?):

Dense vector scores are cosine similarities bound to $[0, 1]$, whereas BM25 scores are unbounded term frequencies $[0, 25+]$. Adding them directly is mathematically invalid. RRF is scale-invariant: it ranks documents strictly by their relative placement in each search list, guaranteeing that chunks appearing high on both lists rise to the top.

• WHAT IS THE NEXT STEP & WHY:

Next Step: Deduplication & Max-Scaled Reranking (Step 7). Why? Fused candidates may contain redundant chunks, and RRF rankings should be rescored with a cross-attention reranker for maximum precision.

STEP 07 Deduplication & Max-Scaled Score Calibration

backend/retrieval/reranker.py • Max-Scaling

• WHAT HAPPENS IN THIS STEP:

Top candidate chunks are deduplicated by chunk ID and text similarity. A Cross-Encoder / scoring model rescores the remaining candidates. **Critical Solved Innovation:** Raw scores are normalized using *Max-Scaling*:

```
normalized_score = raw_score / max(raw_scores)
```

Chunks meeting the `score_threshold >= 0.25` pass to the generator.

• WHY MAX-SCALING (WHY NOT STANDARD MIN-MAX NORMALIZATION?):

Min-max normalization: $(s - \min) / (\max - \min)$ mathematically forces the lowest candidate chunk to \$0.0\$. In a two-chunk document, Chunk 2 always becomes \$0.0\$ and gets dropped by the threshold filter. This was causing second-page project skills to disappear! **Max-Scaling** maintains relative strength and allows all relevant secondary chunks to enter the prompt.

• WHAT IS THE NEXT STEP & WHY:

Next Step: Grounded LLM Generation (Step 8). Why? With the highest-quality candidate chunks in hand, we now instruct the LLM to generate a factual, cited response.

STEP 08 Grounded LLM Streaming Generation (SSE)

Ollama • Qwen 2.5 (1.5B) • Starlette SSE

• WHAT HAPPENS IN THIS STEP:

Retrieved chunks are formatted into a structured prompt with **expanded 3,500-character windows**. The prompt enforces strict rules:

- Answer ONLY from provided text sources.
- Cite every factual assertion with inline bracket tags: `[1]`, `[2]`.
- If unmentioned, state: *"This is not mentioned in the document."*

Tokens stream in real time to the browser via Server-Sent Events (`event: token`).

• WHICH LLM WE USE & WHY:

Ollama with Qwen 2.5 (1.5B) (`qwen2.5:1.5b`).

- **Why Qwen 2.5?** Exceptional instruction-following, structured citations, multilingual support, and 32k context window.
- **Why 1.5B Parameters?** Runs at 40+ tokens/second on consumer CPU/GPUs with only ~1.8GB VRAM footprint. Zero cloud API fees.

• WHY 3,500-CHARACTER SNIPPETS (WHY NOT 900 CHARS?):

In initial testing, `snippet = text[:900]` cut off Chunk 0 right in the middle of the "Technical Skills" table at "Oracle", causing Machine Learning, Generative AI, and tools to vanish. Expanding to 3,500 chars ensures complete sections enter the LLM context.

• WHAT IS THE NEXT STEP & WHY:

Next Step: Best-Box Scoring Refinement (Step 9). Why? The LLM has generated an answer, but we must pinpoint the single exact bounding box to highlight on the PDF canvas.

**STEP
09****Global Best-Box Scoring & Answer
Refinement**

```
refine_citations_for_answer() in  
backend/llm/generator.py
```

• WHAT HAPPENS IN THIS STEP:

Once generation finishes, the backend parses the generated answer text and extracts key entities (numbers, percentages, skill keywords). It computes an overlap score against the raw text of every bounding box across all candidate chunks.

The single highest-scoring bounding box is selected as the winner. All other boxes are pruned. The backend emits

```
event: citations
```

• WHY WE USE GLOBAL BEST-BOX SCORING (WHY NOT ALL CHUNK BOXES?):

Passing all chunk bounding boxes causes the **"Blue Screen" Over-Highlighting Bug**—where 20+ boxes cover an entire page in blue rectangles. Global best-box scoring guarantees single-line precision: only the exact sentence proving the answer lights up.

• WHAT IS THE NEXT STEP & WHY:

Next Step: Synchronized PDF Canvas Highlighting (Step 10). Why? The frontend has received the winning coordinates and must render them onto the PDF canvas.

STEP 10**Synchronized PDF.js Canvas Highlighting**

```
React 18 • PDF.js (3.11) • HTML5 Canvas
```

• WHAT HAPPENS IN THIS STEP:

The right panel opens the PDF via PDF.js on an HTML5 `<canvas>`, navigates to the cited page, scales the normalized coordinates to match the canvas dimensions, and draws an animated bounding box with glowing keyframes:

```
animation: pulseHighlight 1.5s infinite;
```

• WHY WE CONVERT COORDINATE ORIGINS:

PDF documents natively measure coordinates from the **bottom-left**, whereas HTML5 canvas measures from the **top-left**. The frontend checks `box.coord_origin` and dynamically transforms the Y-axis:

```
top = box.coord_origin === "BOTTOMLEFT" ? (pageHeight - box.y1) * scaleY : box.y0 * scaleY;
```

3 Engineering Decision & Tool Selection Matrix

Pipeline Stage	Chosen Technology	Rejected Alternative	Engineering Rationale & Benefit
Parsing	IBM Docling + PyMuPDF	pypdf / pdfminer	Extracts vision layout, markdown tables, and word-level coordinates; standard tools flatten text and lose coordinates.
Chunking	StructureChunker (Custom)	LangChain RecursiveSplitter	Preserves bounding box tuples, table rows, and heading hierarchy; LangChain strips all coordinates making highlights impossible.
Dense Vectors	FastEmbed (BGE-Small)	sentence-transformers (PyTorch)	Pure ONNX runtime in C++, < 5 ms inference, zero PyTorch footprint; #1 MTEB retrieval performance.
Sparse Index	BM25Plus	BM25Okapi	Guarantees non-negative IDF on small corpora ($N \leq 5$); BM25Okapi produces negative scores and drops candidate chunks.
Fusion	Reciprocal Rank Fusion	Raw Score Averaging	Scale-invariant fusion ($k=60$); combines disparate metric scales (cosine similarity vs unbounded BM25 frequencies).
Reranking	Max-Scaling Normalizer	Min-Max Normalizer	Min-max forces 2nd chunk to 0.0, causing secondary sections to drop off. Max-scaling ($s/\max s$) preserves true relative score.
Highlighting	Answer-to-BBox Refiner	Pass All Chunk BBoxes	Evaluates token overlap to pick the 1 exact answer box; completely eliminates the 20-box "blue screen" bug.
Local LLM	Ollama Qwen 2.5 (1.5B)	Cloud APIs (GPT-4 / Claude)	100% private, 40+ tokens/sec, runs locally on 8GB RAM laptops, zero subscription costs.